

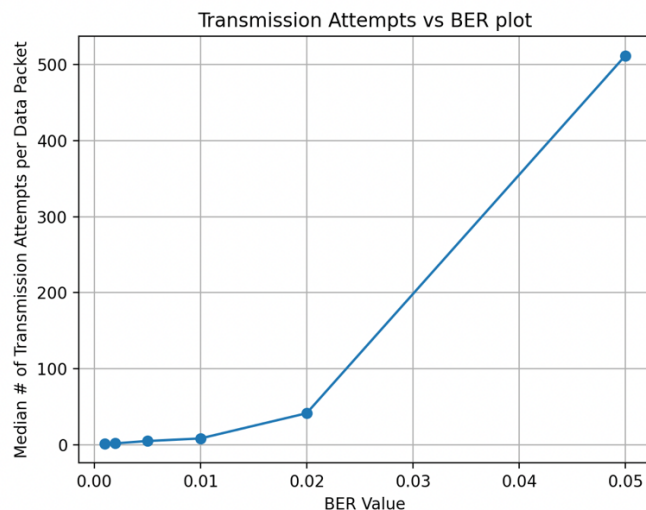
CPR E 4890: Computer Networking and Data Communication

Lab Experiment #5 – or Detection and Go-Back-N ARQ Protocol

Summary

In this lab, I was able to implement the sender's side component for the Go-Back-N ARQ protocol on top of UDP in C code. Basically, the sender maintains a sliding window of size 3, allowing up to three packets to be in transmission at the same time before requiring an acknowledgement. The `introduce_bit_error` function simulated some form of channel noise by randomly flipping bits at a given bit error rate, which basically then forced the protocol to handle corrupted packets.

I also was able to gain a better understanding of how cumulative ACKs and NAKs drive the Go-Back-N retransmission logic. For example, when the receiver detects a corrupted or out of order packet, it sends a NAK or re-ACKs the last good sequence number, which causes the sender to retransmit its entire window from the base value. Observing the output at different BER values made it clear how error rate affects the throughput where at low BERs the window slides forward efficiently, while higher error rates cause more frequent full-window retransmissions.



For the above, most of my values were consistent because I just used the default seed for `rand` and it was not exactly randomized. Hence the results are straightforward and does not have much variation.

AI USAGE:

I used AI for the `plot.py` to ask it why my `matplotlib` was having issues. Turns out I needed to run `python3 -m pip install matplotlib` for my code to work. I have not worked with python as much so I needed help from AI. Apart from that I asked theoretical questions to the AI regarding this assignment such as “Would my performance increase or decrease if my window was bigger/smaller and how does it relate to the error introduced?”